

Azure Deployment Architecture - Explained for Colleagues

Overview

This document explains how the Cultivate Learning ML MVP is deployed on Azure, including the Static Web App (SWA), Container Apps, and the demo architecture.

Current Deployment Architecture

1. Static Web App (SWA) - Frontend Hosting

What it is: Azure Static Web Apps hosts our demo interfaces (Demo 1 and Demo 2) as static HTML/JavaScript files.

Location: <https://calm-tree-06f328310.1.azurestaticapps.net>

Key Components: - **Demo 1:** ML classification demo at </demo1/index.html> - **Demo 2:** Video analysis with Whisper at </demo2/index.html> - **Shared Resources:** ML models and utilities at </shared/>

How it works: 1. Code pushed to `main` branch triggers GitHub Actions 2. GitHub Actions deploys the `unified-demos/` folder to Azure SWA 3. SWA serves static files globally through Azure's CDN 4. Configuration in `staticwebapp.config.json` handles routing and security

2. Container Apps - ML API Backend

What it is: Azure Container Apps hosts our FastAPI ML service for real-time predictions.

API Endpoint: <https://cultivate-ml-api.ashysky-fe559536.eastus.azurecontainerapps.io>

Key Features: - Auto-scales from 0 to 20 instances based on load - Handles ML predictions for OEQ/CEQ classification - Docker-based deployment from Azure Container Registry - Serverless pricing (pay only when running)

How it works: 1. FastAPI service packaged in Docker container 2. Container pushed to Azure Container Registry (ACR) 3. Container Apps pulls image and manages scaling 4. CORS configured to allow SWA frontend to call API

3. Azure Blob Storage - Media & Models

What it is: Cloud storage for videos, ML models, and large files.

Storage Account: `cultivatemlstorage`

Containers: - `videos` - Demo videos and user uploads - `models` - PyTorch/TensorFlow model files - `data` - Training data and results

Project Structure

```
cultivate-uw-ml-mvp/
├── unified-demos/           # Frontend deployment root
│   ├── index.html          # Main landing page
│   ├── demo1/              # Demo 1: ML Classification
│   │   └── index.html       # OEQ/CEQ classifier interface
│   ├── demo2/              # Demo 2: Video Analysis
│   │   ├── index.html       # Whisper transcription demo
│   │   └── warren_teaching_demo_data.json
│   ├── shared/              # Shared JavaScript modules
│   └── staticwebapp.config.json # SWA routing config
├── src/
│   ├── api/                 # FastAPI backend
│   │   └── main.py           # ML API endpoints
│   └── services/             # Azure service integrations
│       └── azure_blob_service.py
├── docker/
│   ├── Dockerfile.api-light # Container App image
│   └── docker-compose.yml    # Local development
└── .github/workflows/
    └── azure-deployment.yml  # CI/CD pipeline
```

Deployment Process

Automatic Deployment (CI/CD)

When you push to the `main` branch:

1. **GitHub Actions triggers** (`.github/workflows/azure-deployment.yml`)
2. **Two parallel jobs run:**
 - **API Deployment:** Builds Docker image → Pushes to ACR → Updates Container App
 - **Frontend Deployment:** Deploys `unified-demos/` folder to SWA

Manual Deployment Commands

```
# Deploy frontend to SWA
az staticwebapp deploy \
  --name cultivate-ml-demos \
  --app-location unified-demos

# Deploy API to Container Apps
docker build -f docker/Dockerfile.api-light -t cultivate-ml-api .
docker tag cultivate-ml-api cultivatemlregistry.azurecr.io/cultivate-ml-api:latest
docker push cultivatemlregistry.azurecr.io/cultivate-ml-api:latest
az containerapp update --name cultivate-ml-api --image cultivatemlregistry.azurecr.io/cultivate-m
```

Demo Deployments Explained

Demo 1: ML Classification System

Purpose: Real-time classification of educational questions (OEQ vs CEQ)

Architecture:

User Browser → SWA (demo1/index.html) → Container App API → ML Model → Response

Flow: 1. User enters text in the web interface 2. JavaScript sends POST request to Container App API 3. FastAPI processes text through PyTorch model 4. Returns classification and confidence scores 5. UI displays results with visual feedback

Demo 2: Video Analysis with Whisper

Purpose: Transcribe and analyze educational videos using OpenAI Whisper

Architecture:

User Upload → Blob Storage → Processing Queue → Whisper API → Results Display

Current Implementation: - Static demo with pre-processed data (`warren_teaching_demo_data.json`) - Shows actual Whisper transcription results - Upload interface ready for backend integration

Future Enhancement: - Real-time video processing using Azure Functions - GPU-enabled Container Instances for Whisper model - Queue-based architecture for scalability

Security & Configuration

Environment Variables

Set in Azure Portal under each service's Configuration:

Container App: - `AZURE_STORAGE_CONNECTION_STRING` - Blob storage access - `MODEL_PATH` - Location of ML models - `API_KEY` - Service authentication

Static Web App: - `API_HOST` - Backend API endpoint - `API_PORT` - HTTPS port (443)

CORS Configuration

Defined in `staticwebapp.config.json`: - Allows API calls from SWA to Container Apps - Enables media streaming from Blob Storage - Implements security headers (CSP, XSS protection)

Monitoring & Troubleshooting

Key Metrics to Monitor

1. SWA Performance:

- Page load times (target: <2s)
- CDN hit ratio (target: >90%)
- 4xx/5xx error rates

2. Container App Health:

- Response times (target: <500ms)
- Scale-out events
- Memory/CPU usage

3. Blob Storage:

- Bandwidth usage
- Request rates
- Storage capacity

Common Issues & Solutions

Frontend not updating: - Check GitHub Actions logs - Verify SWA deployment token in GitHub Secrets - Clear browser cache

API calls failing: - Check CORS settings in `staticwebapp.config.json` - Verify Container App is running (may scale to 0) - Check API logs in Azure Portal

Slow performance: - Review Container App scaling rules - Check if hitting rate limits - Analyze Application Insights metrics

Cost Optimization

Current Setup (Estimated Monthly)

- **Static Web App:** Free tier (100GB bandwidth included)
- **Container Apps:** ~\$30-50 (consumption-based scaling)
- **Blob Storage:** ~\$5-10 (depending on usage)
- **Total:** ~\$35-60/month

Cost Saving Tips

1. Container Apps scale to 0 when idle
2. Use lifecycle policies on Blob Storage
3. Enable caching in SWA configuration
4. Monitor and set spending alerts

Quick Access Links

Production URLs

- **Main Demo Hub:** <https://calm-tree-06f328310.1.azurestaticapps.net>
- **Demo 1 (ML Classification):** <https://calm-tree-06f328310.1.azurestaticapps.net/demo1/>
- **Demo 2 (Video Analysis):** <https://calm-tree-06f328310.1.azurestaticapps.net/demo2/>
- **API Health Check:** <https://cultivate-ml-api.ashysky-fe559536.eastus.azurecontainerapps.io/health>

Azure Portal Resources

- **Resource Group:** cultivate-ml-rg
- **Static Web App:** cultivate-ml-demos
- **Container App:** cultivate-ml-api

- **Container Registry:** cultivatemlregistry
- **Storage Account:** cultivatemlstorage

Summary for Your Colleague

The Cultivate Learning ML MVP uses a modern serverless architecture on Azure:

1. **Static Web App (SWA)** serves the demo interfaces globally with automatic SSL and CDN
2. **Container Apps** provides scalable ML API backend that scales to zero when not in use
3. **Blob Storage** handles videos and large files
4. **GitHub Actions** automates the entire deployment process

The system is designed for: - **High availability** (99.95% SLA) - **Global scale** (CDN distribution)
- **Cost efficiency** (serverless, pay-per-use) - **Easy maintenance** (push to main = automatic deployment)

When explaining to colleagues, emphasize that this is a production-ready, enterprise-grade architecture that can scale from prototype to millions of users without architectural changes.

Created by: Warren & Claude **Date:** November 2025 **Branch:** feature-azure-deploy-docs